Part 1: SQL Benchmark Dataset Preparation and Evaluation Design

Question 1: List of products

Query:

Query | Query History

```
1  SELECT * FROM products;
```

Output:

Data Output | Messages | Notifications

Showing rows: 1 to 110 | Page No: 1 of 1

	productCode [PK] character varying (15)	productName character varying (70)	productLine character varying (50)	productScale character varying (10)	productVendor character varying (50)	productDescription text
1	S10_1678	1969 Harley Davidson Ultimate Chopper	Motorcycles	1:10	Min Lin Diecast	This replica features working kickstand, front s
2	S10_1949	1952 Alpine Renault 1300	Classic Cars	1:10	Classic Metal Creations	Turnable front wheels; steering function; detaile
3	S10_2016	1996 Moto Guzzi 1100i	Motorcycles	1:10	Highway 66 Mini Classi...	Official Moto Guzzi logos and insignias, saddle
4	S10_4698	2003 Harley-Davidson Eagle Drag Bike	Motorcycles	1:10	Red Start Diecast	Model features, official Harley Davidson logos
5	S10_4757	1972 Alfa Romeo GTA	Classic Cars	1:10	Motor City Art Classics	Features include: Turnable front wheels; steerin
6	S10_4962	1962 LanciaA Delta 16V	Classic Cars	1:10	Second Gear Diecast	Features include: Turnable front wheels; steerin
7	S12_1099	1968 Ford Mustang	Classic Cars	1:12	Autoart Studio Design	Hood, doors and trunk all open to reveal highly
8	S12_1108	2001 Ferrari Enzo	Classic Cars	1:12	Second Gear Diecast	Turnable front wheels; steering function; detaile
9	S12_1666	1958 Setra Bus	Trucks and Buses	1:12	Welly Diecast Productio...	Model features 30 windows, skylights & glare re
10	S12_2823	2002 Suzuki XREO	Motorcycles	1:12	Unimax Art Galleries	Official logos and insignias, saddle bags locate
11	S12_3148	1969 Corvair Monza	Classic Cars	1:18	Welly Diecast Productio...	1:18 scale die-cast about 10 long doors open, h
12	S12_3380	1968 Dodge Charger	Classic Cars	1:12	Welly Diecast Productio...	1:12 scale model of a 1968 Dodge Charger. Hoo
13	S12_3891	1969 Ford Falcon	Classic Cars	1:12	Second Gear Diecast	Turnable front wheels; steering function; detaile
14	S12_3990	1970 Plymouth Hemi Cuda	Classic Cars	1:12	Studio M Art Models	Very detailed 1970 Plymouth Cuda model in 1:1
15	S12_4473	1957 Chevy Pickup	Trucks and Buses	1:12	Exoto Designs	1:12 scale die-cast about 20 long Hood opens

Explanation:

This query retrieves every column and row from the products table. It returns all product information including productCode, productName, productLine, productScale, productVendor, productDescription, quantityInStock, buyPrice, and MSRP.

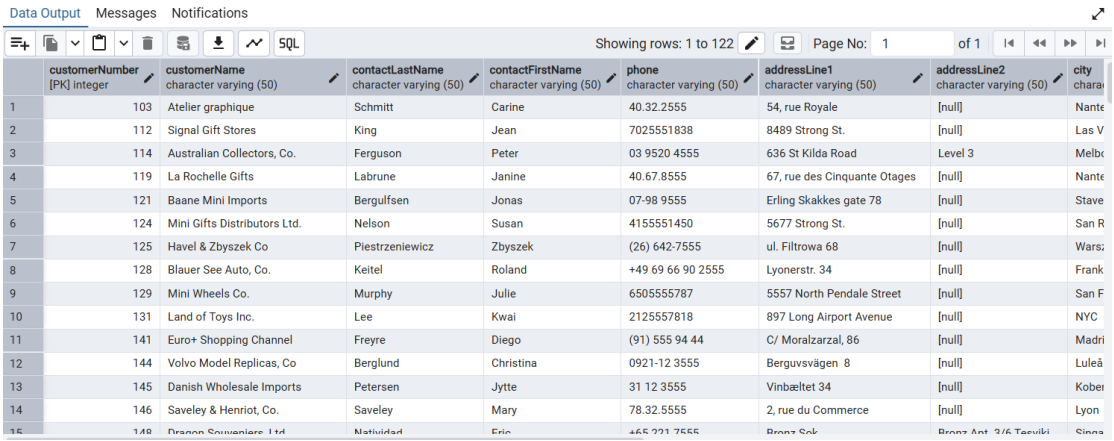
Question 2: Get all customers

Query:

Query | Query History

```
1  SELECT *
2  FROM Customers;
```

Output:



Data Output | Messages | Notifications

Showing rows: 1 to 122 | Page No: 1 of 1

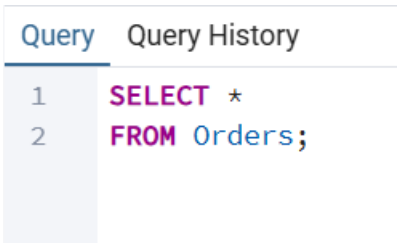
	customerNumber [PK] integer	customerName character varying (50)	contactLastName character varying (50)	contactFirstName character varying (50)	phone character varying (50)	addressLine1 character varying (50)	addressLine2 character varying (50)	city
1	103	Atelier graphique	Schmitt	Carine	40.32.2555	54, rue Royale	[null]	Nante
2	112	Signal Gift Stores	King	Jean	7025551838	8489 Strong St.	[null]	Las V
3	114	Australian Collectors, Co.	Ferguson	Peter	03 9520 4555	636 St Kilda Road	Level 3	Melbo
4	119	La Rochelle Gifts	Labrune	Janine	40.67.8555	67, rue des Cinquante Otages	[null]	Nante
5	121	Baane Mini Imports	Bergulfsen	Jonas	07-98 9555	Erling Skakkes gate 78	[null]	Stave
6	124	Mini Gifts Distributors Ltd.	Nelson	Susan	4155551450	5677 Strong St.	[null]	San R
7	125	Havel & Zbyszek Co	Piestrzeniewicz	Zbyszek	(26) 642-7555	ul. Filtrowa 68	[null]	Wars
8	128	Blauer See Auto, Co.	Keitel	Roland	+49 69 66 90 2555	Lyonerstr. 34	[null]	Frank
9	129	Mini Wheels Co.	Murphy	Julie	6505555787	5557 North Pendale Street	[null]	San F
10	131	Land of Toys Inc.	Lee	Kwai	2125557818	897 Long Airport Avenue	[null]	NYC
11	141	Euro+ Shopping Channel	Freyre	Diego	(91) 555 94 44	C/ Moralzarzal, 86	[null]	Madri
12	144	Volvo Model Replicas, Co	Berglund	Christina	0921-12 3555	Berguvsvägen 8	[null]	Luleå
13	145	Danish Wholesale Imports	Petersen	Jytte	31 12 3555	Vinbæltet 34	[null]	Kober
14	146	Saveley & Henriot, Co.	Saveley	Mary	78.32.5555	2, rue du Commerce	[null]	Lyon
15	148	Dragon Souveniers, Ltd.	Natividad	Eric	+65 221 7555	Bronz Sok	Bronz Apt. 3/6 Tesvikiy	Singa

Explanation:

This query retrieves all customer records from the customers table. It includes customerNumber, customerName, contactLastName, contactFirstName, phone, city, country, creditLimit, and salesRepEmployeeNumber.

Question 3: Show all orders

Query:



Query | Query History

```
1  SELECT *
2  FROM Orders;
```

Output:

Data Output Messages Notifications

Showing rows: 1 to 326 Page No: 1 of 1

	orderNumber [PK] integer	orderDate date	requiredDate date	shippedDate date	status character varying (15)	comments text
1	10100	2003-01-06	2003-01-13	2003-01-10	Shipped	[null]
2	10101	2003-01-09	2003-01-18	2003-01-11	Shipped	Check on availability.
3	10102	2003-01-10	2003-01-18	2003-01-14	Shipped	[null]
4	10103	2003-01-29	2003-02-07	2003-02-02	Shipped	[null]
5	10104	2003-01-31	2003-02-09	2003-02-01	Shipped	[null]
6	10105	2003-02-11	2003-02-21	2003-02-12	Shipped	[null]
7	10106	2003-02-17	2003-02-24	2003-02-21	Shipped	[null]
8	10107	2003-02-24	2003-03-03	2003-02-26	Shipped	Difficult to negotiate with customer. We need more marketing materials
9	10108	2003-03-03	2003-03-12	2003-03-08	Shipped	[null]
10	10109	2003-03-10	2003-03-19	2003-03-11	Shipped	Customer requested that FedEx Ground is used for this shipping
11	10110	2003-03-18	2003-03-24	2003-03-20	Shipped	[null]
12	10111	2003-03-25	2003-03-31	2003-03-30	Shipped	[null]
13	10112	2003-03-24	2003-04-03	2003-03-29	Shipped	Customer requested that ad materials (such as posters, pamphlets) be included in the shippment
14	10113	2003-03-26	2003-04-02	2003-03-27	Shipped	[null]
15	10114	2003-04-01	2003-04-07	2003-04-02	Shipped	[null]

Explanation:

Retrieves all order records. Orders contain fields like orderNumber, orderDate, requiredDate, shippedDate, status, comments, and customerNumber.

Question 4: List all employees

Query:

Query Query History

```
1  SELECT *
2  FROM Employees;
```

Output:

Data Output Messages Notifications

Showing rows: 1 to 23 Page No: 1 of 1

	employeeNumber [PK] integer	lastName character varying (50)	firstName character varying (50)	extension character varying (10)	email character varying (100)	officeCode character varying (10)	reportsTo integer	jobTitle character varying (50)
1	1002	Murphy	Diane	x5800	dmurphy@classicmodelcars.com	1	[null]	President
2	1056	Patterson	Mary	x4611	mpatterso@classicmodelcars.co...	1	1002	VP Sales
3	1076	Firrelli	Jeff	x9273	jfirrelli@classicmodelcars.com	1	1002	VP Marketing
4	1088	Patterson	William	x4871	wpatterson@classicmodelcars.c...	6	1056	Sales Manager (APAC)
5	1102	Bondur	Gerard	x5408	gbondur@classicmodelcars.com	4	1056	Sale Manager (EMEA)
6	1143	Bow	Anthony	x5428	abow@classicmodelcars.com	1	1056	Sales Manager (NA)
7	1165	Jennings	Leslie	x3291	ljennings@classicmodelcars.com	1	1143	Sales Rep
8	1166	Thompson	Leslie	x4065	lthompson@classicmodelcars.c...	1	1143	Sales Rep
9	1188	Firrelli	Julie	x2173	jfirrelli@classicmodelcars.com	2	1143	Sales Rep
10	1216	Patterson	Steve	x4334	spatterson@classicmodelcars.c...	2	1143	Sales Rep
11	1286	Tseng	Foon Yue	x2248	ftseng@classicmodelcars.com	3	1143	Sales Rep
12	1323	Vanauf	George	x4102	gvanauf@classicmodelcars.com	3	1143	Sales Rep
13	1337	Bondur	Loui	x6493	lbondur@classicmodelcars.com	4	1102	Sales Rep
14	1370	Hernandez	Gerard	x2028	ghernande@classicmodelcars.c...	4	1102	Sales Rep
15	1401	Castillo	Pamela	x2759	pcastillo@classicmodelcars.com	4	1102	Sales Rep

Explanation:

Shows all employee records.

Question 5: Get all offices

Query:

Query | Query History

```
1  SELECT *
2  FROM Offices;
```

Output:

Data Output | Messages | Notifications

Showing rows: 1 to 7 | Page No: 1 of 1

	officeCode [PK] character varying (10)	city character varying (50)	phone character varying (50)	addressLine1 character varying (50)	addressLine2 character varying (50)	state character varying (50)	country character varying (50)	postalCode character varyir
1	1	San Francisco	+1 650 219 4782	100 Market Street	Suite 300	CA	USA	94080
2	2	Boston	+1 215 837 0825	1550 Court Place	Suite 102	MA	USA	02107
3	3	NYC	+1 212 555 3000	523 East 53rd Street	apt. 5A	NY	USA	10022
4	4	Paris	+33 14 723 4404	43 Rue Jouffroy Dabba...	[null]	[null]	France	75017
5	5	Tokyo	+81 33 224 5000	4-1 Kioicho	[null]	Chiyoda-Ku	Japan	102-8578
6	6	Sydney	+61 2 9264 2451	5-11 Wentworth Avenue	Floor #2	[null]	Australia	NSW 2010
7	7	London	+44 20 7877 2041	25 Old Broad Street	Level 7	[null]	UK	EC2N 1HN

Explanation:

This query retrieves all office records from the offices table. It includes officeCode, city, phone, addressLine1, addressLine2, state, country, postalCode, and territory.

Question 6: Show all product lines

Query:

Query | Query History

```
1  SELECT *
2  FROM productlines;
```

Output:

Data Output | Messages | Notifications

Showing rows: 1 to 7 | Page No: 1 of 1

	productLine [PK] character varying (50)	textDescription character varying (4000)
1	Classic Cars	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you are looking for classic muscle cars, dream sports cars or movie-inspired miniatures, y
2	Motorcycles	Our motorcycles are state of the art replicas of classic as well as contemporary motorcycle legends such as Harley Davidson, Ducati and Vespa. Models contain stunning details s
3	Planes	Unique, diecast airplane and helicopter replicas suitable for collections, as well as home, office or classroom decorations. Models contain stunning details such as official logos a
4	Ships	The perfect holiday or anniversary gift for executives, clients, friends, and family. These handcrafted model ships are unique, stunning works of art that will be treasured for genera
5	Trains	Model trains are a rewarding hobby for enthusiasts of all ages. Whether you are looking for collectible wooden trains, electric streetcars or locomotives, youwill find a number of gr
6	Trucks and Buses	The Truck and Bus models are realistic replicas of buses and specialized trucks produced from the early 1920s to present. The models range in size from 1:12 to 1:50 scale and in
7	Vintage Cars	Our Vintage Car models realistically portray automobiles produced from the early 1900s through the 1940s. Materials used include Bakelite, diecast, plastic and wood. Most of the

Explanation:

This query retrieves all product line categories from the productlines table.

Question 7: List all payments

Query:

Query | Query History

```
1  SELECT *
2  FROM payments;
```

Output:

Data Output | Messages | Notifications

	customerNumber [PK] integer	checkNumber [PK] character varying (50)	paymentDate date	amount numeric (10,2)
1	103	HQ336336	2004-10-19	6066.78
2	103	JM555205	2003-06-05	14571.44
3	103	OM314933	2004-12-18	1676.14
4	112	BO864823	2004-12-17	14191.12
5	112	HQ55022	2003-06-06	32641.98
6	112	ND748579	2004-08-20	33347.88
7	114	GG31455	2003-05-20	45864.03
8	114	MA765515	2004-12-15	82261.22
9	114	NP603840	2003-05-31	7565.08
10	114	NR27552	2004-03-10	44894.74
11	119	DB933704	2004-11-14	19501.82
12	119	LN373447	2004-08-08	47924.19
13	119	NG94694	2005-02-22	49523.67
14	121	DB889831	2003-02-16	50218.95
15	121	FD317790	2003-10-28	1491.38

Explanation:

This query retrieves all payment records from the payments table. It includes customerNumber, checkNumber, paymentDate, and amount.

Question 8: Get product names and prices

Query:

Query | Query History

```
1  SELECT
2      "productName",
3      "buyPrice",
4      "MSRP"
5  FROM products
6  ORDER BY "productName";
```

Output:

Data Output | Messages | Notifications

	productName character varying (70)	buyPrice numeric (10,2)	MSRP numeric (10,2)
1	18th century schooner	82.34	122.89
2	18th Century Vintage Horse Carriage	60.74	104.72
3	1900s Vintage Bi-Plane	34.25	68.51
4	1900s Vintage Tri-Plane	36.23	72.45
5	1903 Ford Model A	68.30	136.59
6	1904 Buick Runabout	52.66	87.77
7	1911 Ford Town Car	33.30	60.54
8	1912 Ford Model T Delivery Wagon	46.91	88.51
9	1913 Ford Model T Speedster	60.78	101.31
10	1917 Grand Touring Sedan	86.70	170.00
11	1917 Maxwell Touring Car	57.54	99.21
12	1926 Ford Fire Engine	24.92	60.77
13	1928 British Royal Navy Airplane	66.74	109.42
14	1928 Ford Phaeton Deluxe	33.02	68.79

Explanation:

This query retrieves productName together with buyPrice and MSRP. The results are sorted alphabetically by productName.

Question 9: Get customer names and cities

Query:

Query | Query History

```
1  SELECT
2      "customerName",
3      "city",
4      "country"
5  FROM customers
6  ORDER BY "country", "city";
7
```

Output:

Data Output | Messages | Notifications

	customerName character varying (50)	city character varying (50)	country character varying (50)
1	Souveniers And Things Co.	Chatswood	Australia
2	Australian Collectables, Ltd	Glen Waverly	Australia
3	Australian Collectors, Co.	Melbourne	Australia
4	Anna's Decorations, Ltd	North Sydney	Australia
5	Australian Gift Network, Co	South Brisbane	Australia
6	Mini Auto Werke	Graz	Austria
7	Salzburg Collectables	Salzburg	Austria
8	Petit Auto	Bruxelles	Belgium
9	Royale Belge	Charleroi	Belgium
10	Québec Home Shopping Network	Montréal	Canada
11	Royal Canadian Collectables, Ltd.	Tsawassen	Canada
12	Canadian Gift Exchange Network	Vancouver	Canada
13	Heintze Collectables	Århus	Denmark
14	Danish Wholesale Imports	Kobenhavn	Denmark

Explanation:

This query retrieves customerName together with city and country. The results are sorted by country and city.

Question 10: List employee first and last names

Query:

Query | Query History

```sql
SELECT
    "firstName",
    "lastName",
    "jobTitle"
FROM employees
ORDER BY "lastName", "firstName";
```

Output:

Data Output | Messages | Notifications

	firstName character varying (50)	lastName character varying (50)	jobTitle character varying (50)
1	Gerard	Bondur	Sale Manager (EMEA)
2	Loui	Bondur	Sales Rep
3	Larry	Bott	Sales Rep
4	Anthony	Bow	Sales Manager (NA)
5	Pamela	Castillo	Sales Rep
6	Jeff	Firrelli	VP Marketing
7	Julie	Firrelli	Sales Rep
8	Andy	Fixter	Sales Rep
9	Martin	Gerard	Sales Rep
10	Gerard	Hernandez	Sales Rep
11	Leslie	Jennings	Sales Rep
12	Barry	Jones	Sales Rep
13	Yoshimi	Kato	Sales Rep
14	Tom	King	Sales Rep

Explanation:

This query retrieves firstName, lastName, and jobTitle from the employees table. The results are sorted alphabetically.

Question 11: Get all order dates

Query:

Query | Query History

```
SELECT
    "orderNumber",
    "orderDate",
    "requiredDate",
    "shippedDate",
    "status"
FROM orders
ORDER BY "orderDate";
```

Output:

Data Output | Messages | Notifications

	orderNumber [PK] integer	orderDate date	requiredDate date	shippedDate date	status character varying (15)
1	10100	2003-01-06	2003-01-13	2003-01-10	Shipped
2	10101	2003-01-09	2003-01-18	2003-01-11	Shipped
3	10102	2003-01-10	2003-01-18	2003-01-14	Shipped
4	10103	2003-01-29	2003-02-07	2003-02-02	Shipped
5	10104	2003-01-31	2003-02-09	2003-02-01	Shipped
6	10105	2003-02-11	2003-02-21	2003-02-12	Shipped
7	10106	2003-02-17	2003-02-24	2003-02-21	Shipped
8	10107	2003-02-24	2003-03-03	2003-02-26	Shipped
9	10108	2003-03-03	2003-03-12	2003-03-08	Shipped
10	10109	2003-03-10	2003-03-19	2003-03-11	Shipped
11	10110	2003-03-18	2003-03-24	2003-03-20	Shipped

Explanation:

This query retrieves orderNumber, orderDate, requiredDate, shippedDate, and status from the orders table. It helps track order history.

Question 12: Show product vendor list

Query:

Query | Query History

```
1  SELECT DISTINCT "productVendor"
2  FROM products
3  ORDER BY "productVendor";
```

Output:

Data Output | Messages | Notifications

	productVendor character varying (50)
1	Autoart Studio Design
2	Carousel DieCast Legen...
3	Classic Metal Creations
4	Exoto Designs
5	Gearbox Collectibles
6	Highway 66 Mini Classi...
7	Min Lin Diecast
8	Motor City Art Classics
9	Red Start Diecast
10	Second Gear Diecast
11	Studio M Art Models

Explanation:

This query retrieves unique productVendor values using DISTINCT. Duplicate vendor names are removed.

Question 13: Get all product codes

Query:

Query | Query History

```
1  SELECT
2      "productCode",
3      "productName"
4  FROM products
5  ORDER BY "productCode";
```

Output:

Data Output | Messages | Notifications

	productCode [PK] character varying (15)	productName character varying (70)
1	S10_1678	1969 Harley Davidson Ultimate Chopper
2	S10_1949	1952 Alpine Renault 1300
3	S10_2016	1996 Moto Guzzi 1100i
4	S10_4698	2003 Harley-Davidson Eagle Drag Bike
5	S10_4757	1972 Alfa Romeo GTA
6	S10_4962	1962 LanciaA Delta 16V
7	S12_1099	1968 Ford Mustang
8	S12_1108	2001 Ferrari Enzo
9	S12_1666	1958 Setra Bus
10	S12_2823	2002 Suzuki XREO
11	S12_3148	1969 Corvair Monza

Explanation:

This query retrieves productCode together with productName from the products table.

Question 14: List all countries from offices

Query:

Query | Query History

```
1  SELECT DISTINCT "country"
2  FROM offices
3  ORDER BY "country";
```

Output:

Data Output | Messages | Notifications

	country character varying (50)
1	Australia
2	France
3	Japan
4	UK
5	USA

Explanation:

This query retrieves distinct country values from the offices table. It shows all countries where offices are located.

Question 15: Show all order statuses

Query:

Query | Query History

```
1  SELECT DISTINCT "status"
2  FROM orders
3  ORDER BY "status";
```

Output:

Data Output | Messages | Notifications

	status character varying (15)
1	Cancelled
2	Disputed
3	In Process
4	On Hold
5	Resolved
6	Shipped

Explanation:

This query retrieves all unique status values from the orders table using DISTINCT.

Question 16: Get all payment amounts

Query:

Query | Query History

```
SELECT
"customerNumber",
"checkNumber",
"paymentDate",
"amount"
FROM payments
ORDER BY "amount" DESC;
```

Output:

Data Output | Messages | Notifications

	customerNumber [PK] integer	checkNumber [PK] character varying (50)	paymentDate date	amount numeric (10,2)
1	141	JE105477	2005-03-18	120166.58
2	141	ID10962	2004-12-31	116208.40
3	124	KI131716	2003-08-15	111654.40
4	148	KM172879	2003-12-26	105743.00
5	124	AE215433	2005-03-05	101244.59
6	321	DJ15149	2003-11-03	85559.12
7	124	BG255406	2004-08-28	85410.87
8	167	GN228846	2003-12-03	85024.46
9	124	ET64396	2005-04-16	83598.04
10	114	MA765515	2004-12-15	82261.22
11	239	NO865547	2004-03-15	80375.24

Explanation:

This query retrieves customerNumber, checkNumber, paymentDate, and amount from the payments table. Results are sorted by amount in descending order.

Question 17: List all job titles

Query:

Query | Query History

```
1  SELECT DISTINCT "jobTitle"
2  FROM employees
3  ORDER BY "jobTitle";
```

Output:

Data Output | Messages | Notifications

	jobTitle character varying (50)
1	President
2	Sale Manager (EMEA)
3	Sales Manager (APAC)
4	Sales Manager (NA)
5	Sales Rep
6	VP Marketing
7	VP Sales

Explanation:

This query retrieves all unique jobTitle values from the employees table using DISTINCT.

Question 18: Get customer phone numbers

Query:

Query | Query History

```
1  SELECT
2  "customerName",
3  "contactFirstName",
4  "contactLastName",
5  "phone"
6  FROM customers
7  ORDER BY "customerName";
```

Output:

Data Output | Messages | Notifications

Showing rc

	customerName character varying (50)	contactFirstName character varying (50)	contactLastName character varying (50)	phone character varying (50)
1	Alpha Cognac	Annette	Roulet	61.77.6555
2	American Souvenirs Inc	Keith	Franco	2035557845
3	Amica Models & Co.	Paolo	Accorti	011-4988555
4	ANG Resellers	Alejandra	Camino	(91) 745 6555
5	Anna's Decorations, Ltd	Anna	O'Hara	02 9936 8555
6	Anton Designs, Ltd.	Carmen	Anton	+34 913 728555
7	Asian Shopping Network, Co	Brydey	Walker	+612 9411 1555
8	Asian Treasures, Inc.	Patricia	McKenna	2967 555
9	Atelier graphique	Carine	Schmitt	40.32.2555
10	Australian Collectables, Ltd	Sean	Clenahan	61-9-3844-6555
11	Australian Collectors, Co	Peter	Ferguson	03 9520 4555

Explanation:

This query retrieves customerName, contactFirstName, contactLastName, and phone from the customers table.

Question 19: Show product MSRP values

Query:

Query | Query History

```
SELECT
    "productName",
    "MSRP",
    "buyPrice",
    ROUND(("MSRP" - "buyPrice") / "MSRP" * 100, 2) AS margin_pct
FROM products
ORDER BY "MSRP" DESC;
```

Output:

Data Output | Messages | Notifications

	productName character varying (70)	MSRP numeric (10,2)	buyPrice numeric (10,2)	margin_pct numeric
1	1952 Alpine Renault 1300	214.30	98.58	54.00
2	2001 Ferrari Enzo	207.80	95.59	54.00
3	1968 Ford Mustang	194.57	95.34	51.00
4	2003 Harley-Davidson Eagle Drag Bike	193.66	91.02	53.00
5	1969 Ford Falcon	173.02	83.05	52.00
6	1917 Grand Touring Sedan	170.00	86.70	49.00
7	1992 Ferrari 360 Spider red	169.34	77.90	54.00
8	1928 Mercedes-Benz SSK	168.75	72.56	57.00
9	1998 Chrysler Plymouth Prowler	163.73	101.51	38.00
10	1980s Black Hawk Helicopter	157.69	77.27	51.00
11	1969 Corvair Monza	151.08	89.14	41.00

Explanation:

This query retrieves productName, MSRP, and buyPrice and calculates margin_pct for each product.

Question 20: List order numbers

Query:

Query | Query History

```
SELECT
    "orderNumber",
    "orderDate",
    "status"
FROM orders
ORDER BY "orderNumber";
```

Output:

Data Output | Messages | Notifications

	orderNumber [PK] integer	orderDate date	status character varying (15)
1	10100	2003-01-06	Shipped
2	10101	2003-01-09	Shipped
3	10102	2003-01-10	Shipped
4	10103	2003-01-29	Shipped
5	10104	2003-01-31	Shipped
6	10105	2003-02-11	Shipped
7	10106	2003-02-17	Shipped
8	10107	2003-02-24	Shipped
9	10108	2003-03-03	Shipped
10	10109	2003-03-10	Shipped
11	10110	2003-03-18	Shipped

Explanation:

This query retrieves orderNumber, orderDate, and status from the orders table. Results are sorted by orderNumber.

Question 21: Get orders with customer names

Query:

Query | Query History

```sql
SELECT
    o."orderNumber",
    o."orderDate",
    o."status",
    c."customerName",
    c."country"
FROM orders AS o
INNER JOIN customers AS c
    ON o."customerNumber" = c."customerNumber"
ORDER BY o."orderDate" DESC;
```

Output:

Data Output | Messages | Notifications

Showing rows: 1

	orderNumber integer	orderDate date	status character varying (15)	customerName character varying (50)	country character varying (50)
1	10424	2005-05-31	In Process	Euro+ Shopping Channel	Spain
2	10425	2005-05-31	In Process	La Rochelle Gifts	France
3	10423	2005-05-30	In Process	Petit Auto	Belgium
4	10422	2005-05-30	In Process	Diecast Classics Inc.	USA
5	10420	2005-05-29	In Process	Souveniers And Things Co.	Australia
6	10421	2005-05-29	In Process	Mini Gifts Distributors Ltd.	USA
7	10419	2005-05-17	Shipped	Salzburg Collectables	Austria
8	10418	2005-05-16	Shipped	Extreme Desk Decorations, Ltd	New Zealand
9	10417	2005-05-13	Disputed	Euro+ Shopping Channel	Spain
10	10416	2005-05-10	Shipped	L'ordine Souveniers	Italy
11	10415	2005-05-09	Disputed	Australian Collectables, Ltd	Australia

Explanation:

This query joins orders and customers using customerNumber. It retrieves orderNumber, orderDate, status, customerName, and country.

Question 22: Get employees with office city

Query:

Query | Query History

```
SELECT
    e."firstName",
    e."lastName",
    e."jobTitle",
    o."city",
    o."country"
FROM employees AS e
INNER JOIN offices AS o
    ON e."officeCode" = o."officeCode"
ORDER BY o."city", e."lastName";
```

Output:

Data Output | Messages | Notifications

Showing rows: 1 to 23

	firstName character varying (50)	lastName character varying (50)	jobTitle character varying (50)	city character varying (50)	country character varying (50)
1	Julie	Firrelli	Sales Rep	Boston	USA
2	Steve	Patterson	Sales Rep	Boston	USA
3	Larry	Bott	Sales Rep	London	UK
4	Barry	Jones	Sales Rep	London	UK
5	Foon Yue	Tseng	Sales Rep	NYC	USA
6	George	Vanauf	Sales Rep	NYC	USA
7	Loui	Bondur	Sales Rep	Paris	France
8	Gerard	Bondur	Sale Manager (EMEA)	Paris	France
9	Pamela	Castillo	Sales Rep	Paris	France
10	Martin	Gerard	Sales Rep	Paris	France
11	Gerard	Hernandez	Sales Rep	Paris	France

Explanation:

This query joins employees and offices using officeCode. It retrieves firstName, lastName, jobTitle, city, and country.

Question 23: Get payments with customer names

Query:

Query | Query History

```
SELECT
    c."customerName",
    p."checkNumber",
    p."paymentDate",
    p."amount"
FROM payments AS p
INNER JOIN customers AS c
    ON p."customerNumber" = c."customerNumber"
ORDER BY p."amount" DESC;
```

Output:

Data Output | Messages | Notifications

	customerName character varying (50)	checkNumber character varying (50)	paymentDate date	amount numeric (10,2)
1	Euro+ Shopping Channel	JE105477	2005-03-18	120166.58
2	Euro+ Shopping Channel	ID10962	2004-12-31	116208.40
3	Mini Gifts Distributors Ltd.	KI131716	2003-08-15	111654.40
4	Dragon Souveniers, Ltd.	KM172879	2003-12-26	105743.00
5	Mini Gifts Distributors Ltd.	AE215433	2005-03-05	101244.59
6	Corporate Gift Ideas Co.	DJ15149	2003-11-03	85559.12
7	Mini Gifts Distributors Ltd.	BG255406	2004-08-28	85410.87
8	Herkku Gifts	GN228846	2003-12-03	85024.46
9	Mini Gifts Distributors Ltd.	ET64396	2005-04-16	83598.04
10	Australian Collectors, Co.	MA765515	2004-12-15	82261.22
11	Collectable Mini Designs Co	NO865547	2004-03-15	80375.24

Explanation:

This query joins payments and customers using customerNumber. It retrieves customerName, checkNumber, paymentDate, and amount.

Question 24: Get order details with product names

Query:

Query | Query History

```
SELECT
    od."orderNumber",
    p."productName",
    od."quantityOrdered",
    od."priceEach",
    ROUND(od."quantityOrdered" * od."priceEach", 2) AS lineTotal
FROM orderdetails AS od
INNER JOIN products AS p
    ON od."productCode" = p."productCode"
ORDER BY od."orderNumber", od."orderLineNumber";
```

Output:

Data Output | Messages | Notifications

Showing rows:

	orderNumber integer	productName character varying (70)	quantityOrdered integer	priceEach numeric (10,2)	linetotal numeric
1	10100	1936 Mercedes Benz 500k Roadster	49	35.29	1729.21
2	10100	1911 Ford Town Car	50	55.09	2754.50
3	10100	1917 Grand Touring Sedan	30	136.00	4080.00
4	10100	1932 Alfa Romeo 8C2300 Spider Sport	22	75.46	1660.12
5	10101	1928 Mercedes-Benz SSK	26	167.06	4343.56
6	10101	1938 Cadillac V-16 Presidential Limousine	46	44.35	2040.10
7	10101	1939 Chevrolet Deluxe Coupe	45	32.53	1463.85
8	10101	1932 Model A Ford J-Coupe	25	108.06	2701.50
9	10102	1936 Mercedes-Benz 500K Special Road...	41	43.13	1768.33
10	10102	1937 Lincoln Berline	39	95.55	3726.45
11	10103	1962 Volkswagen Microbus	36	107.34	3864.24

Explanation:

This query joins orderdetails and products using productCode. It retrieves orderNumber, productName, quantityOrdered, priceEach, and calculates lineTotal.

Question 25: Get products with product line description

Query:

Query | Query History

```
SELECT
    p."productName",
    p."productLine",
    p."buyPrice",
    p."MSRP",
    pl."textDescription"
FROM products AS p
INNER JOIN productlines AS pl
    ON p."productLine" = pl."productLine"
ORDER BY p."productLine", p."productName";
```

Output:

Data Output | Messages | Notifications

Showing rows: 1 to 110 | Page No: 1 of 1

	productName character varying (70)	productLine character varying (50)	buyPrice numeric (10,2)	MSRP numeric (10,2)	textDescription character varying (4000)
1	1948 Porsche 356-A Roadster	Classic Cars	53.90	77.00	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you ar
2	1948 Porsche Type 356 Roadster	Classic Cars	62.16	141.28	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you ar
3	1949 Jaguar XK 120	Classic Cars	47.25	90.87	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you ar
4	1952 Alpine Renault 1300	Classic Cars	98.58	214.30	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you ar
5	1952 Citroen-15CV	Classic Cars	72.82	117.44	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you ar
6	1956 Porsche 356A Coupe	Classic Cars	98.30	140.43	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you ar
7	1957 Corvette Convertible	Classic Cars	69.93	148.80	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you ar
8	1957 Ford Thunderbird	Classic Cars	34.21	71.27	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you ar
9	1958 Chevy Corvette Limited Edition	Classic Cars	15.91	35.36	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you ar
10	1961 Chevrolet Impala	Classic Cars	32.33	80.84	Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you ar

Explanation:

This query joins products and productlines using productLine. It retrieves productName, productLine, buyPrice, MSRP, and textDescription.

Question 26: Get customers with sales rep names

Query:

Query | Query History

```
SELECT
    c."customerName",
    c."country",
    e."firstName" || ' ' || e."lastName" AS salesRep,
    e."email"
FROM customers AS c
LEFT JOIN employees AS e
    ON c."salesRepEmployeeNumber" = e."employeeNumber"
ORDER BY salesRep NULLS LAST, c."customerName";
```

Output:

Data Output | Messages | Notifications

Showing rows:

	customerName character varying (50)	country character varying (50)	salesrep text	email character varying (100)
1	Anna's Decorations, Ltd	Australia	Andy Fixter	afixter@classicmodelcars.com
2	Australian Collectables, Ltd	Australia	Andy Fixter	afixter@classicmodelcars.com
3	Australian Collectors, Co.	Australia	Andy Fixter	afixter@classicmodelcars.com
4	Australian Gift Network, Co	Australia	Andy Fixter	afixter@classicmodelcars.com
5	Souveniers And Things Co.	Australia	Andy Fixter	afixter@classicmodelcars.com
6	Baane Mini Imports	Norway	Barry Jones	bjones@classicmodelcars.com
7	Bavarian Collectables Imports, ...	Germany	Barry Jones	bjones@classicmodelcars.com
8	Blauer See Auto, Co.	Germany	Barry Jones	bjones@classicmodelcars.com
9	Clover Collections, Co.	Ireland	Barry Jones	bjones@classicmodelcars.com
10	Herkku Gifts	Norway	Barry Jones	bjones@classicmodelcars.com
11	Norway Gifts By Mail, Co	Norway	Barry Jones	bjones@classicmodelcars.com

Explanation:

This query joins customers and employees using salesRepEmployeeNumber and employeeNumber. It retrieves customerName, country, salesRep, and email.

Question 27: Get orders with customer city

Query:

Query | Query History

```sql
SELECT
    o."orderNumber",
    o."orderDate",
    o."status",
    c."customerName",
    c."city",
    c."country"
FROM orders AS o
INNER JOIN customers AS c
    ON o."customerNumber" = c."customerNumber"
ORDER BY c."country", c."city";
```

Output:

Data Output Messages Notifications

Showing rows: 1 to 326

	orderNumber integer	orderDate date	status character varying (15)	customerName character varying (50)	city character varying (50)	country character varying (50)
1	10361	2004-12-17	Shipped	Souveniers And Things Co.	Chatswood	Australia
2	10139	2003-07-16	Shipped	Souveniers And Things Co.	Chatswood	Australia
3	10270	2004-07-19	Shipped	Souveniers And Things Co.	Chatswood	Australia
4	10420	2005-05-29	In Process	Souveniers And Things Co.	Chatswood	Australia
5	10193	2003-11-21	Shipped	Australian Collectables, Ltd	Glen Waverly	Australia
6	10265	2004-07-02	Shipped	Australian Collectables, Ltd	Glen Waverly	Australia
7	10415	2005-05-09	Disputed	Australian Collectables, Ltd	Glen Waverly	Australia
8	10125	2003-05-21	Shipped	Australian Collectors, Co.	Melbourne	Australia
9	10347	2004-11-29	Shipped	Australian Collectors, Co.	Melbourne	Australia
10	10120	2003-04-29	Shipped	Australian Collectors, Co.	Melbourne	Australia
11	10342	2004-11-24	Shipped	Australian Collectors, Co.	Melbourne	Australia

Explanation:

This query joins orders and customers using customerNumber. It retrieves orderNumber, orderDate, status, customerName, city, and country.

Question 28: Get employees and their manager

Query:

Query Query History

```
1  SELECT
2      e."firstName" || ' ' || e."lastName" AS employee,
3      e."jobTitle",
4      m."firstName" || ' ' || m."lastName" AS manager,
5      m."jobTitle" AS managerTitle
6  FROM employees AS e
7  LEFT JOIN employees AS m
8      ON e."reportsTo" = m."employeeNumber"
9  ORDER BY manager NULLS FIRST, employee;
```

Output:

Data Output | Messages | Notifications

	employee text	jobTitle character varying (50)	manager text	managertitle character varying (50)
1	Diane Murphy	President	[null]	[null]
2	Foon Yue Tseng	Sales Rep	Anthony Bow	Sales Manager (NA)
3	George Vanauf	Sales Rep	Anthony Bow	Sales Manager (NA)
4	Julie Firrelli	Sales Rep	Anthony Bow	Sales Manager (NA)
5	Leslie Jennings	Sales Rep	Anthony Bow	Sales Manager (NA)
6	Leslie Thompson	Sales Rep	Anthony Bow	Sales Manager (NA)
7	Steve Patterson	Sales Rep	Anthony Bow	Sales Manager (NA)
8	Jeff Firrelli	VP Marketing	Diane Murphy	President
9	Mary Patterson	VP Sales	Diane Murphy	President
10	Barry Jones	Sales Rep	Gerard Bondur	Sale Manager (EMEA)
11	Gerard Hernand	Sales Rep	Gerard Bondur	Sale Manager (EMEA)

Explanation:

This query performs a self-join on the employees table using reportsTo and employeeNumber. It retrieves employee, jobTitle, manager, and managerTitle.

Question 29: Get orderdetails with product vendor

Query:

Query | Query History

```
SELECT
    od."orderNumber",
    p."productVendor",
    p."productName",
    od."quantityOrdered",
    od."priceEach"
FROM orderdetails AS od
INNER JOIN products AS p
    ON od."productCode" = p."productCode"
ORDER BY p."productVendor", od."orderNumber";
```

Output:

Data Output | Messages | Notifications

Showing rows: 1 to 1000

	orderNumber integer	productVendor character varying (50)	productName character varying (70)	quantityOrdered integer	priceEach numeric (10,2)
1	10101	Autoart Studio Design	1932 Model A Ford J-Coupe	25	108.06
2	10103	Autoart Studio Design	1962 Volkswagen Microbus	36	107.34
3	10105	Autoart Studio Design	The Schooner Bluenose	41	54.00
4	10106	Autoart Studio Design	1937 Horch 930V Limousine	31	55.89
5	10106	Autoart Studio Design	1900s Vintage Bi-Plane	49	65.77
6	10107	Autoart Studio Design	1997 BMW R 1100 S	25	96.92
7	10108	Autoart Studio Design	1968 Ford Mustang	33	165.38
8	10108	Autoart Studio Design	2002 Yamaha YZR M1	34	74.85
9	10110	Autoart Studio Design	1932 Model A Ford J-Coupe	33	115.69
10	10114	Autoart Studio Design	1962 Volkswagen Microbus	21	102.23
11	10119	Autoart Studio Design	1900s Vintage Bi-Plane	41	64.40

Explanation:

This query joins orderdetails and products using productCode. It retrieves orderNumber, productVendor, productName, quantityOrdered, and priceEach.

Question 30: Get payments with customer country

Query:

Query | Query History

```
SELECT
    c."country",
    c."customerName",
    p."checkNumber",
    p."paymentDate",
    p."amount"
FROM payments AS p
INNER JOIN customers AS c
    ON p."customerNumber" = c."customerNumber"
ORDER BY c."country", p."amount" DESC;
```

Output:

Data Output | Messages | Notifications

Showing rows: 1 to 2

	country character varying (50)	customerName character varying (50)	checkNumber character varying (50)	paymentDate date	amount numeric (10,2)
1	Australia	Australian Collectors, Co.	MA765515	2004-12-15	82261.22
2	Australia	Australian Collectors, Co.	GG31455	2003-05-20	45864.03
3	Australia	Australian Collectors, Co.	NR27552	2004-03-10	44894.74
4	Australia	Anna's Decorations, Ltd	LE432182	2003-09-28	41554.73
5	Australia	Anna's Decorations, Ltd	KM841847	2003-11-13	38547.19
6	Australia	Souveniers And Things Co.	JT819493	2004-08-02	35806.73
7	Australia	Australian Collectables, Ltd	CO645196	2003-12-10	35505.63
8	Australia	Souveniers And Things Co.	OD327378	2005-01-03	31835.36
9	Australia	Anna's Decorations, Ltd	OJ819725	2005-04-30	29848.52
10	Australia	Anna's Decorations, Ltd	EM979878	2005-02-09	27083.78
11	Australia	Souveniers And Things Co.	IA793562	2003-08-03	24013.52

Explanation:

This query joins payments and customers using customerNumber. It retrieves country, customerName, checkNumber, paymentDate, and amount.

Question 31: Count customers per country

Query:

Query | Query History

```
SELECT
    "country",
    COUNT(*) AS customerCount
FROM customers
GROUP BY "country"
ORDER BY customerCount DESC;
```

Output:

Data Output | Messages | Notifications

	country character varying (50)	customercount bigint
1	USA	36
2	Germany	13
3	France	12
4	Spain	7
5	UK	5
6	Australia	5
7	Italy	4
8	New Zealand	4
9	Switzerland	3
10	Singapore	3
11	Canada	3

Explanation:

This query groups customers by country and counts the number of customers in each country using COUNT(*).

Question 32: Total payments per customer

Query:

Query | Query History

```
SELECT
    c."customerName",
    COUNT(p."checkNumber") AS paymentCount,
    ROUND(SUM(p."amount"), 2) AS totalPaid
FROM payments AS p
INNER JOIN customers AS c
    ON p."customerNumber" = c."customerNumber"
GROUP BY c."customerName"
ORDER BY totalPaid DESC;
```

Output:

Data Output | Messages | Notifications

	customerName character varying (50)	paymentcount bigint	totalpaid numeric
1	Euro+ Shopping Channel	13	715738.98
2	Mini Gifts Distributors Ltd.	9	584188.24
3	Australian Collectors, Co.	4	180585.07
4	Muscle Machine Inc	4	177913.95
5	Dragon Souveniers, Ltd.	4	156251.03
6	Down Under Souveniers, Inc	4	154622.08
7	AV Stores, Co.	3	148410.09
8	Anna's Decorations, Ltd	4	137034.22
9	Corporate Gift Ideas Co.	2	132340.78
10	Saveley & Henriot, Co.	3	130305.35
11	Rovelli Gifts	3	127529.69

Explanation:

This query joins payments and customers using customerNumber. It groups by customerName and calculates paymentCount and totalPaid.

Question 33: Number of orders per status

Query:

Query | Query History

```
SELECT
    "status",
    COUNT(*) AS orderCount,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS pct
FROM orders
GROUP BY "status"
ORDER BY orderCount DESC;
```

Output:

Data Output | Messages | Notifications

	status character varying (15)	ordercount bigint	pct numeric
1	Shipped	303	92.94
2	In Process	6	1.84
3	Cancelled	6	1.84
4	Resolved	4	1.23
5	On Hold	4	1.23
6	Disputed	3	0.92

Explanation:

This query groups orders by status and calculates orderCount and percentage values using COUNT(*) and a window function.

Question 34: Products per product line

Query:

Query | Query History

```
1  SELECT
2      "productLine",
3      COUNT(*) AS productCount
4  FROM products
5  GROUP BY "productLine"
6  ORDER BY productCount DESC;
```

Output:

Data Output Messages Notifications

	productLine character varying (50)	productcount bigint
1	Classic Cars	38
2	Vintage Cars	24
3	Motorcycles	13
4	Planes	12
5	Trucks and Buses	11
6	Ships	9
7	Trains	3

Explanation:

This query groups products by productLine and counts the number of products in each category.

Question 35: Employees per office

Query:

Query Query History

```
SELECT
    o."city",
    o."country",
    COUNT(e."employeeNumber") AS employeeCount
FROM offices AS o
INNER JOIN employees AS e
    ON o."officeCode" = e."officeCode"
GROUP BY o."officeCode", o."city", o."country"
ORDER BY employeeCount DESC;
```

Output:

Data Output | Messages | Notifications

	city character varying (50)	country character varying (50)	employeecount bigint
1	San Francisco	USA	6
2	Paris	France	5
3	Sydney	Australia	4
4	NYC	USA	2
5	London	UK	2
6	Boston	USA	2
7	Tokyo	Japan	2

Explanation:

This query joins employees and offices using officeCode. It groups by officeCode, city, and country to count employees per office.

Question 36: Total stock per product vendor

Query:

Query | Query History

```
SELECT
    "productVendor",
    COUNT(*) AS productCount,
    SUM("quantityInStock") AS totalStock
FROM products
GROUP BY "productVendor"
ORDER BY totalStock DESC;
```

Output:

Data Output | Messages | Notifications

	productVendor character varying (50)	productcount bigint	totalstock bigint
1	Gearbox Collectibles	9	60495
2	Min Lin Diecast	8	50089
3	Classic Metal Creations	10	45408
4	Welly Diecast Productio…	8	45095
5	Exoto Designs	9	44166
6	Motor City Art Classics	9	43105
7	Second Gear Diecast	8	42865
8	Studio M Art Models	8	42253
9	Carousel DieCast Legen…	9	40805
10	Unimax Art Galleries	8	38191
11	Highway 66 Mini Classi	9	37520

Explanation:

This query groups products by productVendor and calculates productCount and totalStock using COUNT(*) and SUM(quantityInStock).

Question 37: Average buy price per product line

Query:

Query | Query History

```
1  SELECT
2      "productLine",
3      COUNT(*) AS productCount,
4      ROUND(AVG("buyPrice"), 2) AS avgBuyPrice,
5      ROUND(MIN("buyPrice"), 2) AS minBuyPrice,
6      ROUND(MAX("buyPrice"), 2) AS maxBuyPrice
7  FROM products
8  GROUP BY "productLine"
9  ORDER BY avgBuyPrice DESC;
```

Output:

Data Output | Messages | Notifications

	productLine character varying (50)	productcount bigint	avgbuyprice numeric	minbuyprice numeric	maxbuyprice numeric
1	Classic Cars	38	64.45	15.91	103.42
2	Trucks and Buses	11	56.33	24.92	84.76
3	Motorcycles	13	50.69	24.14	91.02
4	Planes	12	49.63	29.34	77.27
5	Ships	9	47.01	33.30	82.34
6	Vintage Cars	24	46.07	20.61	86.70
7	Trains	3	43.92	26.72	67.56

Explanation:

This query groups products by productLine and calculates avgBuyPrice, minBuyPrice, and maxBuyPrice.

Question 38: Orders per customer

Query:

Query | Query History

```
SELECT
    c."customerName",
    c."country",
    COUNT(o."orderNumber") AS orderCount
FROM orders AS o
INNER JOIN customers AS c
    ON o."customerNumber" = c."customerNumber"
GROUP BY c."customerNumber", c."customerName", c."country"
ORDER BY orderCount DESC;
```

Output:

Data Output | Messages | Notifications

	customerName character varying (50)	country character varying (50)	ordercount bigint
1	Euro+ Shopping Channel	Spain	26
2	Mini Gifts Distributors Ltd.	USA	17
3	Dragon Souveniers, Ltd.	Singapore	5
4	Reims Collectables	France	5
5	Down Under Souveniers, Inc	New Zealand	5
6	Australian Collectors, Co.	Australia	5
7	Danish Wholesale Imports	Denmark	5
8	Baane Mini Imports	Norway	4
9	Tokyo Collectables, Ltd	Japan	4
10	La Rochelle Gifts	France	4
11	Blauer See Auto, Co.	Germany	4

Explanation:

This query joins orders and customers using customerNumber. It groups by customerName and country to calculate orderCount.

Question 39: Max MSRP per product line

Query:

Query | Query History

```
SELECT
    "productLine",
    ROUND(MAX("MSRP"), 2) AS maxMSRP,
    ROUND(AVG("MSRP"), 2) AS avgMSRP
FROM products
GROUP BY "productLine"
ORDER BY maxMSRP DESC;
```

Output:

Data Output Messages Notifications

	productLine character varying (50)	maxmsrp numeric	avgmsrp numeric
1	Classic Cars	214.30	118.02
2	Motorcycles	193.66	97.18
3	Vintage Cars	170.00	87.10
4	Planes	157.69	89.52
5	Trucks and Buses	136.67	103.18
6	Ships	122.89	86.56
7	Trains	100.84	73.85

Explanation:

This query groups products by productLine and calculates maxMSRP and avgMSRP.

Question 40: Min buy price per vendor

Query:

Query Query History

```
SELECT
    "productVendor",
    ROUND(MIN("buyPrice"), 2) AS minBuyPrice,
    ROUND(MAX("buyPrice"), 2) AS maxBuyPrice,
    COUNT(*) AS productCount
FROM products
GROUP BY "productVendor"
ORDER BY minBuyPrice ASC;
```

Output:

Data Output | Messages | Notifications

	productVendor character varying (50)	minbuyprice numeric	maxbuyprice numeric	productcount bigint
1	Carousel DieCast Legen...	15.91	82.34	9
2	Second Gear Diecast	16.24	103.42	8
3	Classic Metal Creations	20.61	98.58	10
4	Red Start Diecast	21.75	91.02	7
5	Motor City Art Classics	22.57	85.68	9
6	Studio M Art Models	23.14	58.33	8
7	Gearbox Collectibles	24.14	101.51	9
8	Welly Diecast Productio...	24.23	89.14	8
9	Autoart Studio Design	26.30	95.34	8
10	Highway 66 Mini Classi...	32.37	83.51	9
11	Unimax Art Galleries	33.30	77.90	8

Explanation:

This query groups products by productVendor and calculates minBuyPrice, maxBuyPrice, and productCount.

Question 41: Total number of customers

Query:

Query | Query History

```
1  SELECT COUNT(*) AS totalCustomers
2  FROM customers;
```

Output:

Data Output | Messages | Notifications

	totalcustomers bigint
1	122

Explanation:

This query counts the total number of rows in the customers table using COUNT(*).

Question 42: Total number of products

Query:

Query | Query History

```
1  SELECT COUNT(*) AS totalProducts
2  FROM products;
```

Output:

Data Output | Messages | Notifications

	totalproducts bigint
1	110

Explanation:

This query counts the total number of rows in the products table using COUNT(*).

Question 43: Total revenue from payments

Query:

Query | Query History

```
1  SELECT
2      COUNT(*) AS totalPayments,
3      ROUND(SUM("amount"), 2) AS totalRevenue,
4      ROUND(AVG("amount"), 2) AS avgPayment
5  FROM payments;
```

Output:

Data Output | Messages | Notifications

	totalpayments bigint	totalrevenue numeric	avgpayment numeric
1	273	8853839.23	32431.65

Explanation:

This query calculates totalPayments, totalRevenue, and avgPayment from the payments table using COUNT(*), SUM(amount), and AVG(amount).

Question 44: Average product price

Query:

Query | Query History

```
1  SELECT
2      ROUND(AVG("buyPrice"), 2) AS avgBuyPrice,
3      ROUND(AVG("MSRP"), 2) AS avgMSRP,
4      ROUND(AVG("MSRP" - "buyPrice"), 2) AS avgMargin
5  FROM products;
```

Output:

Data Output | Messages | Notifications

	avgbuyprice numeric	avgmsrp numeric	avgmargin numeric
1	54.40	100.44	46.04

Explanation:

This query calculates avgBuyPrice, avgMSRP, and avgMargin from the products table using AVG().

Question 45: Max payment amount

Query:

Query | Query History

```
1  SELECT MAX(amount) AS max_payment FROM payments;
```

Output:

Data Output | Messages | Notifications

	max_payment numeric
1	120166.58

Explanation:

This query finds the maximum amount value in the payments table and returns maxPayment.

Question 46: Min payment amount

Query:

Query | Query History

```
1  SELECT MIN(amount) AS min_payment FROM payments;
```

Output:

Data Output | Messages | Notifications

	min_payment numeric
1	615.45

Explanation:

This query finds the minimum amount value in the payments table and returns min_payment.

Question 47: Count total orders

Query:

Query | Query History

```
SELECT
COUNT(*) AS totalOrders,
COUNT(CASE WHEN "status" = 'Shipped' THEN 1 END) AS shipped,
COUNT(CASE WHEN "status" = 'Cancelled' THEN 1 END) AS cancelled
FROM orders;
```

Output:

Data Output | Messages | Notifications

	totalorders bigint	shipped bigint	cancelled bigint
1	326	303	6

Explanation:

This query calculates totalOrders together with shipped and cancelled counts using COUNT(*) and conditional aggregation.

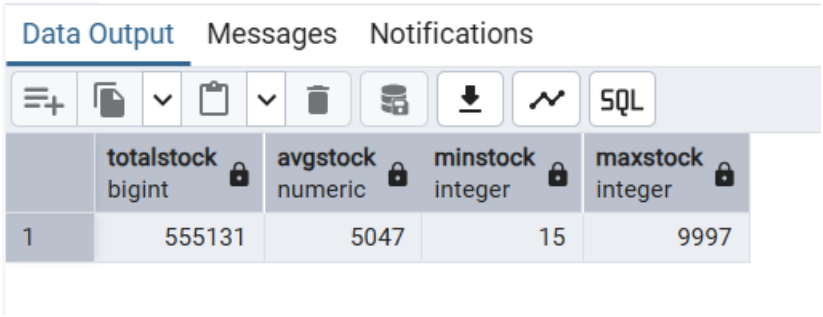
Question 48: Total quantity in stock

Query:

Query | Query History

```
SELECT
SUM("quantityInStock") AS totalStock,
ROUND(AVG("quantityInStock"), 0) AS avgStock,
MIN("quantityInStock") AS minStock,
MAX("quantityInStock") AS maxStock
FROM products;
```

Output:



	totalstock bigint	avgstock numeric	minstock integer	maxstock integer
1	555131	5047	15	9997

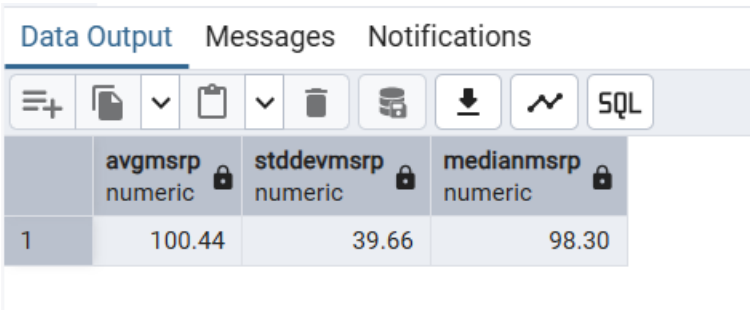
Explanation:

This query calculates totalStock, avgStock, minStock, and maxStock from quantityInStock in the products table.

Question 49: Average MSRP

Query:

Output:



	avgmsrp numeric	stddevmsrp numeric	medianmsrp numeric
1	100.44	39.66	98.30

Explanation:

This query calculates avgMSRP, stddevMSRP, and medianMSRP from the MSRP column using AVG(), STDDEV(), and PERCENTILE_CONT().

Question 50: Number of employees

Query:

Query | Query History

```sql
SELECT
    COUNT(*) AS totalEmployees,
    COUNT(DISTINCT "officeCode") AS officeCount,
    COUNT(DISTINCT "jobTitle") AS uniqueJobTitles
FROM employees;
```

Output:

Data Output | Messages | Notifications

	totalemployees bigint	officecount bigint	uniquejobtitles bigint
1	23	7	7

Explanation:

This query calculates totalEmployees, officeCount, and uniqueJobTitles from the employees table using COUNT(*) and COUNT(DISTINCT).

Part 2: Design an Evaluation Strategy for Text-to-SQL Agents

Introduction

Evaluating a Text-to-SQL system is challenging because multiple correct SQL queries can answer the same natural language question. Two different queries might return identical results even when using different joins, filters, or aggregation methods. Conversely, a query might execute successfully but return incorrect results due to logical errors.

For this reason, evaluation cannot focus solely on whether SQL runs successfully. A comprehensive evaluation framework must measure execution quality, result correctness, schema understanding, efficiency, robustness, and natural language response quality together.

Core Evaluation Metrics

M1. Execution Accuracy (EA)

Execution Accuracy measures whether the generated SQL query runs successfully without syntax errors, runtime failures, or timeouts.

Formula: EA = Successful Executions / Total Queries

How to evaluate: Execute each generated SQL query against the PostgreSQL database. Queries that run successfully receive a pass, while failed queries receive a fail.

Why it matters: A query that cannot execute is unusable even if the intent is correct.

Target: Acceptable performance is above 90%. Production-level systems should achieve at least 98%.

M2. Result Set Match (RSM)

Result Set Match measures whether the generated query returns the same result as the reference query.

Formula: RSM = Correct Result Sets / Total Queries

How to evaluate: Compare the returned rows and values with the ground truth results. Ordering differences are ignored unless ORDER BY is explicitly required.

Why it matters: This is one of the most important metrics because the final output must match the expected answer.

Target: Acceptable performance is above 80%. Production systems should target above 92%.

M3. Structural SQL Accuracy (SSA)

Structural SQL Accuracy checks whether the generated query uses the correct SQL structure, including correct tables, columns, join types, WHERE conditions, and aggregation functions.

How to evaluate: Parse both the generated SQL and reference SQL into syntax trees and compare them structurally.

Why it matters: A query may accidentally return correct results while still using incorrect structure. SSA helps identify such cases.

Target: Acceptable performance is above 75%. Production systems should target above 88%.

M4. Schema Compliance (SC)

Schema Compliance checks whether the query references only valid tables and columns from the database schema.

Formula: SC = Queries Without Invalid References / Total Queries

How to evaluate: Verify all table names and column names against PostgreSQL information_schema metadata.

Why it matters: Hallucinated columns and tables are a major failure type in Text-to-SQL systems.

Target: Acceptable performance is above 95%. Production systems should target above 99%.

M5. Query Efficiency Score (QES)

Query Efficiency evaluates whether the generated SQL avoids unnecessary complexity such as unnecessary JOINs, SELECT * on large tables, cartesian products, redundant subqueries, or missing DISTINCT where required.

How to evaluate: Perform static analysis on the SQL query structure to detect anti-patterns.

Why it matters: Efficient queries improve performance and reduce database load.

Target: This metric is mainly used for optimization analysis rather than strict pass/fail evaluation.

M6. Robustness to Ambiguity (RA)

This metric evaluates how well the agent handles ambiguous natural language questions. For example, "Show top customers" does not specify whether "top" means highest revenue, most orders, or largest payments.

How to evaluate: The benchmark includes intentionally ambiguous questions. Human evaluators score whether the system asks clarifying questions or makes reasonable assumptions on a scale of 0 (poor handling), 0.5 (partially reasonable), to 1 (good handling).

Why it matters: Real-world users often ask incomplete or ambiguous questions.

Target: At least 70% reasonable responses.

M7. Self-Correction Rate (SCR)

Self-Correction Rate measures whether the agent can recover from mistakes after failure.

Formula: SCR = Corrected Queries After Retry / Initially Failed Queries

How to evaluate: If a query fails or produces incorrect results, allow the system up to 3 retry attempts.

Why it matters: Modern AI agents should improve through retry and self-correction rather than failing immediately.

Target: At least 60% of failed queries should be corrected within 3 attempts.

M8. Natural Language Answer Quality (NLAQ)

This metric evaluates the quality of the natural language response returned after query execution, including clarity, accuracy, completeness, and absence of hallucinated information.

How to evaluate: Human evaluators score responses on a scale from 1 to 5.

Why it matters: Users usually prefer readable explanations instead of raw SQL tables.

Target: Average score should be at least 3.5 out of 5.

Composite Evaluation Score

To summarize overall system quality, calculate a weighted composite score:

Composite Score =

- 0.30 × Execution Accuracy
- 0.30 × Result Set Match
- 0.20 × Schema Compliance
- 0.10 × Structural SQL Accuracy
- 0.06 × Self-Correction Rate
- 0.04 × Robustness to Ambiguity

Query Efficiency and Natural Language Quality are tracked separately as additional quality indicators.

Score Interpretation:

- 92–100: Production-ready system suitable for deployment
- 80–91: Good system with minor improvements needed
- 65–79: Fair system with noticeable weaknesses
- Below 65: Poor system requiring major improvement

Evaluation Pipeline

The evaluation process follows these steps:

Step 1: The user provides a natural language question.

Step 2: The Text-to-SQL agent generates an SQL query.

Step 3: Schema Compliance checks verify all tables and columns exist in the database.

Step 4: The SQL query is executed against PostgreSQL.

Step 5: The returned result set is compared with the expected ground truth.

Step 6: The SQL structure is analyzed for correct joins, filters, and aggregations.

Step 7: If execution or results fail, the retry mechanism attempts correction.

Step 8: The natural language response is evaluated for clarity and accuracy.

Step 9: All metric scores are combined into the final evaluation score.

Benchmark Dataset Design

Difficulty Levels

Questions should be divided into three categories. Easy questions involve single-table queries. Medium questions require two-table joins and simple aggregations. Hard questions test complex joins, nested queries, window functions, and subqueries.

Ambiguous Question Set

Include intentionally ambiguous questions to test the system's clarification ability. This reveals whether the agent can identify when information is missing and ask appropriate follow-up questions.

Error Injection Testing

Introduce incorrect schema references in prompts to evaluate hallucination resistance. This tests whether the system validates against the actual schema or invents nonexistent tables and columns.

Paraphrasing Tests

Test the same question using multiple writing styles including formal language, casual language, and passive voice. This measures language understanding robustness across different user communication patterns.

Regression Testing

After every system update, rerun the full benchmark set to detect performance regressions. This ensures that improvements in one area do not degrade performance in another.

Conclusion

A strong Text-to-SQL evaluation framework must measure more than SQL syntax correctness. It should evaluate execution success, result accuracy, schema

understanding, query efficiency, robustness, self-correction ability, and natural language quality together. This framework provides a practical and scalable way to evaluate mini Text-to-SQL agents developed in future assignments and can serve as the foundation for evaluating production-grade AI database assistants.